

Using extra features and descriptors

Contents

- Loading packages and data
- Extra atom features, atom descriptors, bond features
- Extra molecule features

This notebook demonstrates how to use extra features and descriptors in addition to the default Chemprop featurizers.

- Extra atom and bond features are used in addition to those calculated by Chemprop internally.
- Extra atom descriptors get incorporated into the atom descriptors from message passing via a learned linear transformation.
- Extra bond descriptors are not currently supported because the bond descriptors from message passing are not used for molecular property prediction.
- Extra molecule features can be used as extra datapoint descriptors, which are concatenated to the output of the aggregation layer before the final prediction layer.

 [Open in Colab](#)

```
[1]: # Install chemprop from GitHub if running in Google Colab
import os

if os.getenv("COLAB_RELEASE_TAG"):
    try:
        import chemprop
    except ImportError:
        !git clone https://github.com/chemprop/chemprop.git
        %cd chemprop
        !pip install .
        %cd examples
```

Loading packages and data

```
[2]: import numpy as np
import pandas as pd
from pathlib import Path

from lightning import pytorch as pl
from rdkit import Chem

from chemprop import data, featurizers, models, nn, utils

[3]: chemprop_dir = Path.cwd().parent
input_path = chemprop_dir / "tests" / "data" / "regression" / "mol" / "mo
smiles_column = "smiles"
target_columns = ["lipo"]

[4]: df_input = pd.read_csv(input_path)
smis = df_input.loc[:, smiles_column].values
ys = df_input.loc[:, target_columns].values
```

Getting extra features and descriptors

The `rdkit.Chem.Mol` representation of molecules is needed as input to many featurizers. Chemprop provides a helpful wrapper to rdkit to make these from SMILES.

```
[5]: mols = [utils.make_mol(smi, keep_h=False, add_h=False) for smi in smis]
```

Extra atom features, atom descriptors, bond features

Extra atom and bond features frequently come from QM calculations. The calculation results can be saved to a file and then loaded in a notebook using pandas or numpy. The loaded atom or bond features can be a list of numpy arrays where each numpy array of features corresponds to a single molecule in the dataset. Each row in an array corresponds to a different atom or bond in the same order of atoms or bonds in the `rdkit.Chem.Mol` objects.

The atom features could also be used as extra atom descriptors.

```
[6]: # This code is just a placeholder for the actual QM calculation

def QM_calculation(mol):
    n_extra_atom_feats = 10
    n_extra_bond_feats = 4
    extra_atom_features = np.array([np.random.randn(n_extra_atom_feats) for i in range(n_extra_atom_feats)])
    extra_bond_features = np.array([np.random.randn(n_extra_bond_feats) for i in range(n_extra_bond_feats)])
    return extra_atom_features, extra_bond_features

extra_atom_featuress = []
extra_bond_featuress = []

for mol in mols:
    extra_atom_features, extra_bond_features = QM_calculation(mol)
    extra_atom_featuress.append(extra_atom_features)
    extra_bond_featuress.append(extra_bond_features)

# Save to a file
np.savez("atom_features.npz", *extra_atom_featuress)
np.savez("bond_features.npz", *extra_bond_featuress)
```

```
[7]: extra_atom_featuress = np.load("atom_features.npz")
extra_atom_featuress = [extra_atom_featuress[f"arr_{i}"] for i in range(len(extra_atom_featuress))]

extra_atom_descriptorss = extra_atom_featuress

extra_bond_featuress = np.load("bond_features.npz")
extra_bond_featuress = [extra_bond_featuress[f"arr_{i}"] for i in range(len(extra_bond_featuress))]
```

You can also get extra atom and bond features from other sources.

```
[8]: atom_radii = {1: 0.79, 5: 1.2, 6: 0.91, 7: 0.75, 8: 0.65, 9: 0.57, 16: 1.1}

extra_atom_featuress = [
    np.vstack([np.array([atom_radii[a.GetAtomicNum()]] for a in mol.GetAtoms()])
    for mol in mols]
```

Extra molecule features

A QM calculation could also be used to get extra molecule features. Extra molecule features are different from extra atom and bond features in that they are stored in a single numpy array where each row corresponds to a single molecule in the dataset, instead of a list of numpy arrays.

```
[9]: def QM_calculation(mol):
      n_extra_mol_feats = 7
      return np.random.randn(n_extra_mol_feats)

      extra_mol_features = np.array([QM_calculation(mol) for mol in mols])

      np.savez("mol_features.npz", extra_mol_features)
```

```
[10]: extra_mol_features = np.load("mol_features.npz")
```

The extra molecule features can also be calculated using built-in Chemprop featurizers or featurizers from other packages.

```
[11]: molecule_featurizer = featurizers.MorganBinaryFeaturizer()

      extra_mol_features = np.array([molecule_featurizer(mol) for mol in mols])
```

```
[12]: # First install other package
      # !pip install descriptastorus

      # from descriptastorus.descriptors import rdNormalizedDescriptors
      # generator = rdNormalizedDescriptors.RDKit2DNormalized()
      # extra_mol_features = np.array([generator.process(smi)[1:] for smi in smi_list])
```

The molecule featurizers available in Chemprop are registered in

`MoleculeFeaturizerRegistry`.

```
[13]: for MoleculeFeaturizer in featurizers.MoleculeFeaturizerRegistry.keys():
      print(MoleculeFeaturizer)

      morgan_binary
      morgan_count
      rdkit_2d
      v1_rdkit_2d
      v1_rdkit_2d_normalized
      charge
```

If your model takes multiple components as input, you can use extra molecule features for each component as extra datapoint descriptors. Simply concatenate the extra molecule features together before passing them to the datapoints.

```
[14]: extra_mol_features_comp1 = np.random.rand(len(mols), 5)
      extra_mol_features_comp2 = np.random.rand(len(mols), 5)

      extra_datapoint_descriptors = np.concatenate(
          [extra_mol_features_comp1, extra_mol_features_comp2], axis=1
      )
```

Making datapoints, datasets, and dataloaders

Once you have all the extra features and descriptors your model will use, you can make the datapoints.

```
[15]: datapoints = [
        data.MoleculeDatapoint(mol, y, V_f=V_f, E_f=E_f, V_d=V_d, x_d=X_d)
        for mol, y, V_f, E_f, V_d, X_d in zip(
            mols,
            ys,
            extra_atom_featuress,
            extra_bond_featuress,
            extra_atom_descriptorss,
            extra_datapoint_descriptors,
        )
    ]
```

After splitting the data, the datasets are made. To make a dataset, you need a `MolGraph` featurizer, which needs to be told the size of extra atom and bond features.

```
[16]: n_extra_atom_feats = extra_atom_featuress[0].shape[1]
n_extra_bond_feats = extra_bond_featuress[0].shape[1]

featurizer = featurizers.SimpleMoleculeMolGraphFeaturizer(
    extra_atom_fdim=n_extra_atom_feats, extra_bond_fdim=n_extra_bond_feats
)

train_indices, val_indices, test_indices = data.make_split_indices(mols,
train_data, val_data, test_data = data.split_data_by_indices(
    datapoints, train_indices, val_indices, test_indices
)

train_dset = data.MoleculeDataset(train_data[0], featurizer)
val_dset = data.MoleculeDataset(val_data[0], featurizer)
test_dset = data.MoleculeDataset(test_data[0], featurizer)
```

The return type of make_split_indices has changed in v2.1 - see help(make_

Often scaling the extra features and descriptors improves model performance. The scalers for the extra features and descriptors should be fit to the training dataset, applied to the validation dataset, and then given to the model to apply to the test dataset at prediction time. This is the same as for scaling target values to improve model performance.

```
[17]: targets_scaler = train_dset.normalize_targets()
      extra_atom_features_scaler = train_dset.normalize_inputs("V_f")
      extra_bond_features_scaler = train_dset.normalize_inputs("E_f")
      extra_atom_descriptors_scaler = train_dset.normalize_inputs("V_d")
      extra_datapoint_descriptors_scaler = train_dset.normalize_inputs("X_d")

      val_dset.normalize_targets(targets_scaler)
      val_dset.normalize_inputs("V_f", extra_atom_features_scaler)
      val_dset.normalize_inputs("E_f", extra_bond_features_scaler)
      val_dset.normalize_inputs("V_d", extra_atom_descriptors_scaler)
      val_dset.normalize_inputs("X_d", extra_datapoint_descriptors_scaler)
```

```
[17]: ▼ StandardScaler ⓘ ?
      ► Parameters
```

```
[18]: # Featurize the train and val datasets to save computation time.
      train_dset.cache = True
      val_dset.cache = True

      train_loader = data.build_dataloader(train_dset)
      val_loader = data.build_dataloader(val_dset, shuffle=False)
      test_loader = data.build_dataloader(test_dset, shuffle=False)
```

Making the model

The message passing layer needs to know the total size of atom and bond features (i.e. the sum of the sizes of the Chemprop atom and bond features and the extra atom and bond features). The `MolGraph` featurizer collects this information. The message passing layer also needs to know the number of extra atom descriptors.

The extra atom and bond features scalers are combined into a graph transform which is given to the message passing layer to use at prediction time. To avoid scaling the atom and bond features from the internal Chemprop featurizers, the graph transform uses a pad equal to the length of features from the Chemprop internal atom and bond featurizers. This information is stored in the `MolGraph` featurizer.

The extra atom descriptor scaler are also converted to a transform and given to the message passing layer to use at prediction time.

```
[19]: n_V_features = featurizer.atom_fdim - featurizer.extra_atom_fdim
      n_E_features = featurizer.bond_fdim - featurizer.extra_bond_fdim

      V_f_transform = nn.ScaleTransform.from_standard_scaler(extra_atom_features)
      E_f_transform = nn.ScaleTransform.from_standard_scaler(extra_bond_features)

      graph_transform = nn.GraphTransform(V_f_transform, E_f_transform)

      V_d_transform = nn.ScaleTransform.from_standard_scaler(extra_atom_descriptors)
```

```
[20]: n_extra_atom_descs = extra_atom_descriptors[0].shape[1]

      mp = nn.BondMessagePassing(
          d_v=featurizer.atom_fdim,
          d_e=featurizer.bond_fdim,
          d_vd=n_extra_atom_descs,
          graph_transform=graph_transform,
          V_d_transform=V_d_transform,
      )
```

The predictor layer needs to know the size of its input, including any extra datapoint descriptors.

```
[21]: ffn_input_dim = mp.output_dim + extra_datapoint_descriptors.shape[1]

      output_transform = nn.UnscaleTransform.from_standard_scaler(targets_scaler)
      ffn = nn.RegressionFFN(input_dim=ffn_input_dim, output_transform=output_transform)
```

The overall model is given the transform from the extra datapoint descriptors scaler.

```
[22]: X_d_transform = nn.ScaleTransform.from_standard_scaler(extra_datapoint_descriptors)

      chemprop_model = models.MPNN(mp, nn.NormAggregation(), ffn, X_d_transform)
```

Training and prediction

The rest of the training and prediction are the same as other Chemprop workflows.

```
[23]: trainer = pl.Trainer(
        logger=False, enable_checkpointing=False, enable_progress_bar=True,
    )

GPU available: False, used: False
TPU available: False, using: 0 TPU cores
```

```
[24]: trainer.fit(chemprop_model, train_loader, val_loader)

Loading `train_dataloader` to estimate number of stepping batches.
/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightning
```

	Name	Type	Params	Mode	FLOPs
0	message_passing	BondMessagePassing	325 K	train	0
1	agg	NormAggregation	0	train	0
2	bn	Identity	0	train	0
3	predictor	RegressionFFN	96.6 K	train	0
4	X_d_transform	ScaleTransform	0	train	0
5	metrics	ModuleList	0	train	0

Trainable params: 422 K

Non-trainable params: 0

Total params: 422 K

Total estimated model params size (MB): 1

Modules in train mode: 27

Modules in eval mode: 0

Total FLOPs: 0

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightning.pytorch.py:434: The 'val_dataloader' does not have many workers which may be a value of the 'num_workers' argument to 'num_workers=11' in the 'DataLoader'.

'Trainer.fit' stopped: 'max_epochs=5' reached.

```
[25]: results = trainer.test(chemprop_model, test_loader)
```

Test metric	DataLoader 0
test/mse	1.0164769887924194

/home/knathan/anaconda3/envs/chemprop/lib/python3.11/site-packages/lightning.pytorch.py:434: The 'val_dataloader' does not have many workers which may be a value of the 'num_workers' argument to 'num_workers=11' in the 'DataLoader'.